

Rethinking Normalization Placement for LLMs: Post-Norm under Curriculum Depth Growing

Sheng Ren¹, Yadong Wang¹, Naiqiang Tan², Jiangang Kong², Jun Fang²,
Rui Liu², Jun Wang², Kai Chen², Lipeng Liang², Xiang Chen^{1*}

¹Nanjing University of Aeronautics and Astronautics, Nanjing, Jiangsu, China

²Didichuxing Co. Ltd, Beijing, China

{rensheng, xiang_chen}@nuaa.edu.cn

Abstract

Pre-norm is the standard normalization placement in modern Transformers because it facilitates joint optimization of full-depth models. We ask whether this preference persists when depth is introduced through a curriculum. In curriculum depth growth, each appended block receives the boundary representation produced by a trained prefix, making normalization placement relevant to forward conditioning. We therefore test whether placement and training curriculum interact. In a controlled distillation study with a Qwen3-8B teacher and a nine-layer student, pre-norm and post-norm are indistinguishable under joint training, differing by 0.0004 validation CE, while post-norm improves over pre-norm by 0.0328 under curriculum growth, an order of magnitude larger. A post-joint control matched by student active-layer tokens remains worse than post-grow, which rules out compute as the sole explanation. The ranking crosses over during the curriculum: post-norm takes the lead once blocks are appended. Single-block and freeze controls localize the ranking change to block appending rather than shallow-block quality or retraining. Boundary diagnostics associate post-norm with stable residual scales and pre-norm with structural-token scale drift; on a fixed batch, the final pre-grow block is also nearly identity-mapped. Together with the phase-wise crossover, these observations are consistent with boundary-scale conditioning after new blocks are appended. The results motivate treating normalization placement and training curriculum as coupled design choices in this distillation setting.

Introduction

Pre-norm has become the standard choice in modern Transformer language models (Xiong et al. 2020; Touvron et al. 2023). This preference is well motivated in conventional end-to-end training. When a full-depth Transformer is trained from random initialization, normalization before each sub-layer improves gradient behavior at initialization and reduces the need for carefully tuned learning-rate warm-up in practice (Xiong et al. 2020; Liu et al. 2020).

This finding has influenced decoder-only LLMs and distilled Transformer students (Touvron et al. 2023; Elhoushi et al. 2024). However, post-norm remains competitive when its optimization difficulties are addressed (Wang et al. 2024; Ding et al. 2021). Normalization placement may therefore

Where does normalization go?

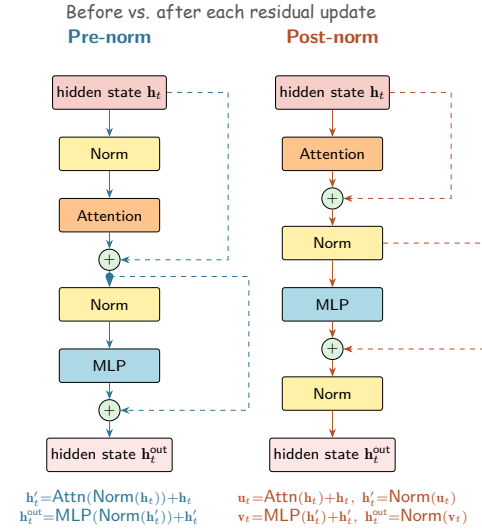


Figure 1: Pre-norm normalizes sub-layer inputs, whereas post-norm normalizes the residual stream after each update.

depend on the training regime rather than on the layer formulation alone, an issue we take up here.

We study this question in curriculum depth growth. Unlike joint training, in which all layers are active from the first update, curriculum depth growth begins with a shallow model and appends new blocks in later phases (Gu et al. 2021; Du et al. 2024). The final model is therefore constructed through a sequence of inherited initializations. At each phase, the blocks trained in the previous phase determine the input distribution received by the newly appended block. This perspective is related to Net2Net-style network growth (Chen, Goodfellow, and Shlens 2016; Wei et al. 2016), but it raises a question that existing growth methods generally leave unanswered: which normalization placement provides a better-conditioned boundary input for a newly added block?

We hypothesize that normalization placement interacts with the way depth is introduced. Under joint training, normalization primarily supports optimization through a complete stack. Under curriculum depth growth, it also shapes

*Corresponding author.

the forward distribution passed from the trained stack to each newly appended block. Pre-norm normalizes sub-layer inputs but does not explicitly normalize the residual stream emitted across block boundaries, whereas post-norm normalizes after each residual update. We therefore expect the placements to behave similarly under joint training but to separate after the curriculum begins appending blocks.

We test this hypothesis in a controlled block-stack distillation setting. The student uses a frozen teacher embedding and a teacher-initialized language-modeling head that is trained jointly with its appendable decoder blocks. Under joint training, the two placements differ by only 0.0004 validation CE; under curriculum growth, post-norm improves over pre-norm by 0.0328 after additional blocks are appended, yielding an interaction of -0.0332 . A post-joint control matched by student active-layer tokens remains worse than post-grow, which rules out additional student-decoder compute as the sole explanation. Block-boundary diagnostics show stable residual scales for post-norm and increasing scale drift for pre-norm, providing correlational evidence for the proposed mechanism. In summary, our contributions are as follows:

- We recast curriculum depth growth as block-wise initialization, shifting the normalization question from full-depth optimization to inherited boundary conditioning.
- We show that normalization placement interacts with the training curriculum: pre-norm and post-norm are nearly tied under joint training, whereas post-norm performs better under sequential depth growth in the evaluated block-stack setting.
- We analyze the interaction through single-block controls, block-removal analysis, and block-boundary activation diagnostics, obtaining converging evidence for forward conditioning.

Related Work

Normalization Placement in Transformers. The original Transformer adopted post-norm (Vaswani et al. 2017), while later analyses showed that pre-norm improves initialization-time gradients and reduces warm-up sensitivity (Xiong et al. 2020; Liu et al. 2020; Ba, Kiros, and Hinton 2016), motivating its use in modern decoder-only LLMs (Touvron et al. 2023; Team 2025). Sandwich norm, DeepNorm, and ReZero stabilize post-norm or related residual designs (Ding et al. 2021; Wang et al. 2024; Bachlechner et al. 2021), while recent work explores placements beyond the standard dichotomy (Kim et al. 2025; Zheng et al. 2026; Loshchilov et al. 2025). HybridNorm combines QKV normalization with post-norm FFNs within each block to capture the strengths of both placements (Zhuo et al. 2025), and SiameseNorm couples pre-norm-like and post-norm-like streams through shared residual blocks to reconcile stability and representational capacity (Li et al. 2026). A principled forward-backward stability analysis of normalization placements further clarifies why different placements induce distinct training dynamics at fixed depth (Kan et al. 2025). Initialization and residual scaling provide another stability lever (Zhang, Dauphin, and Ma 2019; Huang et al. 2020). These studies largely assume fixed-depth end-to-end training; we instead

test whether the preferred placement changes when depth is introduced sequentially. Our objective is therefore different from proposing another fixed-depth stabilization rule: we ask whether the same placement can change rank when the training path changes.

Network Growth and Function Preservation. Progressive growth expands a trained smaller network, often through function-preserving transformations (Chen, Goodfellow, and Shlens 2016; Wei et al. 2016). Transformer variants include StackBERT, stacking operators for LLM pretraining, and block insertion into pretrained models (Gu et al. 2021; Du et al. 2024; Wu et al. 2024). These methods primarily optimize training efficiency or preserve the function of the smaller network, while normalization placement is usually inherited and held fixed. They do not isolate whether placement itself interacts with the joint-versus-grow protocol. We instead view curriculum growth as inherited initialization: an independently parameterized block is trained on the boundary distribution produced by the existing stack, making normalization placement part of the growth condition rather than a fixed inherited choice.

Block-Stack Knowledge Distillation. Transformer distillation typically studies which logits, hidden states, or attention signals a fixed-depth student should match (Hinton, Vinyals, and Dean 2015; Gou et al. 2021; Jiao et al. 2020; Sanh et al. 2019; Sun et al. 2020; Wang et al. 2020). LayerSkip shares the language-modeling head for early exits, while stochastic depth varies active layers without imposing a depth-introduction order (Elhoushi et al. 2024; Huang et al. 2016). Pruning methods also produce compact stacks from the top down (Men et al. 2025; Xia et al. 2024). These approaches determine what a fixed-depth student should retain or which existing layers should be removed, but they do not study how a trained prefix initializes the input distribution of a newly appended block. Our bottom-up block-stack setting adds independently parameterized blocks sequentially and compares normalization placements at the resulting phase boundaries, separating this question from fixed-depth distillation and top-down compression.

Methodology

This section introduces the block-stack student, the curriculum depth-growth protocol, and the forward-conditioning hypothesis underlying our comparison. The Experiments section describes the implementation and evaluates the interaction between normalization placement and the training curriculum. The formulation below makes the training path and block-boundary condition explicit.

Depth-Growable Block-Stack Student

Block-Stack Student. We use a block-stack student whose depth can be expanded during training. Given a teacher Transformer with T decoder layers, the student is divided into K blocks with L decoder layers per block, resulting in a total depth of $S = K \cdot L \ll T$. The vocabulary embedding $\mathbf{E}$ and language-modeling head $\mathbf{W}_{\text{lm}}$ are copied from the teacher. The embedding remains frozen, whereas the

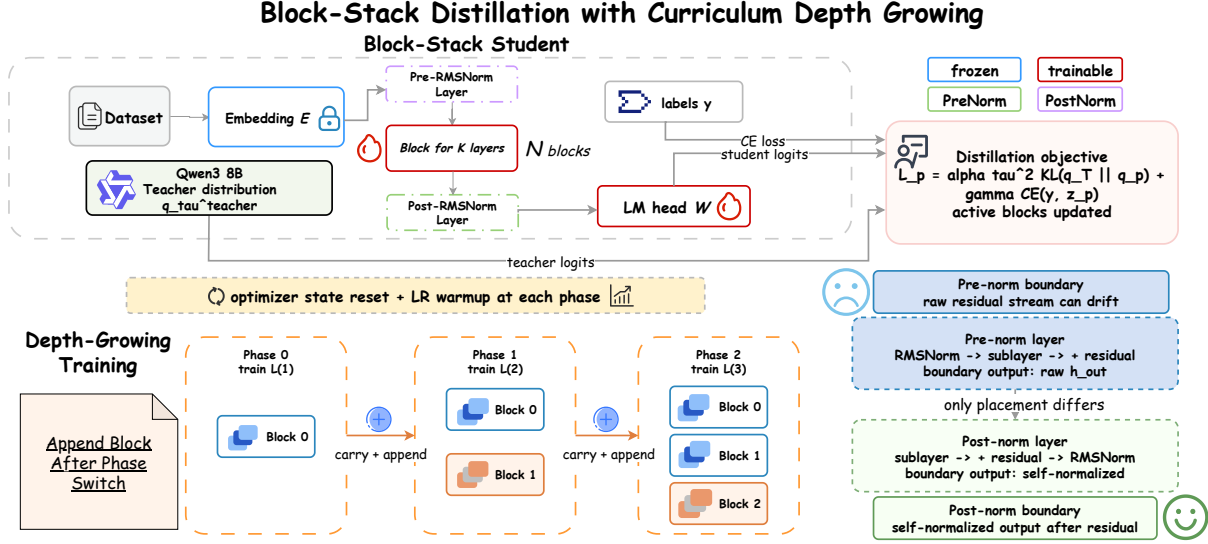


Figure 2: Overview of the block-stack distillation framework. The student uses a frozen teacher embedding and a teacher-initialized, trainable language-modeling head, while its decoder is divided into appendable blocks. Joint training activates all blocks from the beginning, whereas curriculum growth adds one block in each phase. Pre-norm and post-norm differ in whether normalization is applied before or after each residual update.

language-modeling head and the independently parameterized student decoder layers are optimized. This design fixes the input interface and initializes the output interface identically across variants while allowing the decoder stack and language-modeling head to adapt during distillation, consistent with common Transformer distillation settings (Hinton, Vinyals, and Dean 2015; Jiao et al. 2020). Each block contains a contiguous group of decoder layers and, once introduced, remains active in all subsequent phases.

All variants share the same attention and MLP sub-layers—rotary position embeddings (Su et al. 2024), SwiGLU activations (Shazeer 2020), grouped-query attention (Ainslie et al. 2023), and scaled dot-product attention—so the comparison isolates the placement of RMSNorm (Zhang and Sennrich 2019) relative to each residual update.

Pre-Norm and Post-Norm Layers. We compare two placements of RMSNorm. Pre-norm applies RMSNorm immediately before each sub-layer but does not explicitly normalize the residual stream after each residual update:

$$\begin{aligned} \mathbf{h}' &= \text{Attention}(\text{RMSNorm}(\mathbf{h})) + \mathbf{h}, \\ \mathbf{h}_{\text{out}} &= \text{MLP}(\text{RMSNorm}(\mathbf{h}')) + \mathbf{h}'. \end{aligned} \quad (1)$$

Following the standard pre-norm decoder interface, we apply a final RMSNorm before the language-modeling head:

$$\mathbf{h}_{\text{final}} = \mathbf{N}_{\text{final}}(\mathbf{h}_K). \quad (2)$$

Post-norm applies RMSNorm after each residual addition:

$$\begin{aligned} \mathbf{u} &= \text{Attention}(\mathbf{h}) + \mathbf{h}, & \mathbf{h}' &= \text{RMSNorm}(\mathbf{u}), \\ \mathbf{v} &= \text{MLP}(\mathbf{h}') + \mathbf{h}', & \mathbf{h}_{\text{out}} &= \text{RMSNorm}(\mathbf{v}). \end{aligned} \quad (3)$$

Because the final post-norm layer already produces a normalized representation, no additional RMSNorm is applied after the final block. The first block, however, receives the raw embedding $\mathbf{E}(\mathbf{x})$, so we apply a single entry RMSNorm $\mathbf{N}_{\text{entry}}$ before it. Both $\mathbf{N}_{\text{final}}$ (pre-norm) and $\mathbf{N}_{\text{entry}}$ (post-norm) are initialized from the teacher’s final RMSNorm and remain trainable. Both variants therefore contain two RMSNorm modules per decoder layer and exactly one model-boundary RMSNorm, giving them the same number of trainable normalization parameters. Thus, internal block boundaries differ through normalization placement rather than parameter count.

Curriculum Depth Growth

Growth as Inherited Initialization. Curriculum depth growth introduces decoder blocks in phases rather than activating the full stack from the beginning. At phase p , the active student contains the first $p + 1$ blocks, corresponding to $L(p + 1)$ decoder layers. The parameters learned in earlier phases are retained, and one additional block is activated when the model depth increases. Each phase therefore initializes a deeper model from a trained prefix and a newly activated, randomly initialized block.

This protocol is related to progressive network growth and Net2Net-style expansion (Chen, Goodfellow, and Shlens 2016; Wei et al. 2016; Gu et al. 2021; Du et al. 2024; Wu et al. 2024), but it does not require the expanded model to preserve the exact function of the preceding phase. Instead, we study how the trained prefix determines the input distribution received by the newly introduced block during

Algorithm 1: Curriculum Depth-Growth Distillation

Require: Teacher components $(\mathbf{E}, \mathbf{N}, \mathbf{W}_{\text{lm}})$, blocks K , layers per block L , loss weights α, γ , and phase budgets $B_0, \dots, B_{K-1}$.

Ensure: A trained block-stack student.

- 1: Randomly initialize all student decoder blocks.
 - 2: Copy $\mathbf{E}$ and $\mathbf{W}_{\text{lm}}$; freeze $\mathbf{E}$ and initialize the placement-specific boundary norm from $\mathbf{N}$.
 - 3: **for** phase $p = 0$ **to** $K - 1$ **do**
 - 4: Activate the first $p + 1$ blocks.
 - 5: Reset optimizer state and restart the phase LR schedule.
 - 6: **for** step $t = 1$ **to** B_p **do**
 - 7: Obtain teacher and active-student logits.
 - 8: Compute the KD+CE objective $\mathcal{L}_p$.
 - 9: Update all active decoder blocks, the boundary norm, and $\mathbf{W}_{\text{lm}}$.
 - 10: **end for**
 - 11: **end for**
-

subsequent distillation. This boundary distribution is part of the initialization condition faced by the appended block.

Per-Phase Distillation Objective. At each phase, the active student produces

$$\mathbf{h}^{(p)} = \text{Student}(\mathbf{x}; \text{active_blocks} = p + 1), \quad (4)$$

and is trained with the same distillation objective:

$$\mathcal{L}_p = \alpha \tau^2 \text{KL}(q_\tau^{\text{teacher}} \parallel q_\tau^{(p)}) + \gamma \text{CE}(\mathbf{y}, \mathbf{z}^{(p)}), \quad (5)$$

where $q_\tau = \text{softmax}(\mathbf{z}/\tau)$, τ is the distillation temperature, $\mathbf{z}^{(p)}$ denotes the logits of the active student, and $\mathbf{y}$ denotes the next-token labels. All active decoder blocks, the boundary norm, and the language-modeling head are optimized jointly within each phase.

Joint Training as the Comparison Protocol. Under joint training, all K blocks are active from the first optimization step and are trained end to end with the objective in Eq. 5. The two protocols use the same final architecture and objective but differ in staged block activation and, under the prescribed grow protocol, optimizer and learning-rate resets at phase boundaries. The experimental setup details these protocol differences and the token- and compute-matching comparisons.

Placement–Curriculum Interaction. Let $\ell_{m,c}$ denote final held-out CE for placement $m \in \{\text{pre}, \text{post}\}$ and curriculum $c \in \{\text{joint}, \text{grow}\}$. We measure the within-curriculum gap and factorial interaction as

$$\Delta_c = \ell_{\text{post},c} - \ell_{\text{pre},c}, \quad \mathcal{I} = \Delta_{\text{grow}} - \Delta_{\text{joint}}. \quad (6)$$

A negative $\mathcal{I}$ means that post-norm becomes relatively more favorable under curriculum growth.

Forward-Conditioning Hypothesis

Curriculum depth growth creates a boundary condition specific to staged training. When a new block is introduced, it

receives the residual stream produced by the prefix trained in the preceding phase. Normalization placement therefore affects not only computation within each layer but also the forward distribution received by each newly introduced block.

Pre-norm and post-norm handle this boundary distribution differently. Pre-norm applies RMSNorm immediately before the attention and MLP sub-layers but does not explicitly normalize the representation after each residual update. Post-norm applies RMSNorm after every residual update, so the representation passed to the next layer or across a block boundary is explicitly normalized.

We hypothesize that this distinction becomes important during curriculum depth growth. Each newly introduced block is optimized on top of the boundary distribution produced by the existing stack. Explicit normalization of block outputs may therefore provide a more stable input distribution for subsequent phases. We test this hypothesis by comparing pre-norm and post-norm under joint training and curriculum depth growth, controlling for single-block performance and active compute, and examining activation scales and block contributions at block boundaries.

Testable Predictions. The hypothesis yields three observable predictions. First, post-norm need not outperform pre-norm before any block is appended, because the initial shallow model does not yet inherit a trained block boundary. Second, any placement gap should emerge after a depth transition rather than uniformly throughout training. Third, the placement favored by growth should exhibit better-controlled boundary representations and make nontrivial use of the appended blocks. These predictions separate forward conditioning from an unconditional post-norm advantage; the single-block, compute-matched, freeze, activation-scale, and block-removal analyses test the corresponding alternatives and observable consequences.

Experiments

We conduct a series of controlled experiments to investigate whether normalization placement remains a fixed architectural choice when Transformer depth is introduced through a curriculum. Using a block-stack distillation setting, we compare pre-norm and post-norm students under joint training and curriculum depth growth. The primary comparison matches the teacher, architecture, objective, data shard, and unique-token exposure; the curriculum protocol additionally changes active depth, optimization steps, data replay, and phase resets, with compute examined separately. We ask whether placement changes the joint-grow ranking (**RQ1**), whether single-block quality or active compute explains the grow advantage (**RQ2**), and whether boundary dynamics support forward conditioning as a contributing mechanism (**RQ3**).

Experimental Setup

Teacher, Student, and Data. We use the base Qwen3-8B model (Team 2025) as the teacher and train a block-stack student with $K=3$ blocks and $L=3$ decoder layers per block, giving $S=9$ trainable layers in total. Following the block-stack distillation setting (Hinton, Vinyals, and Dean

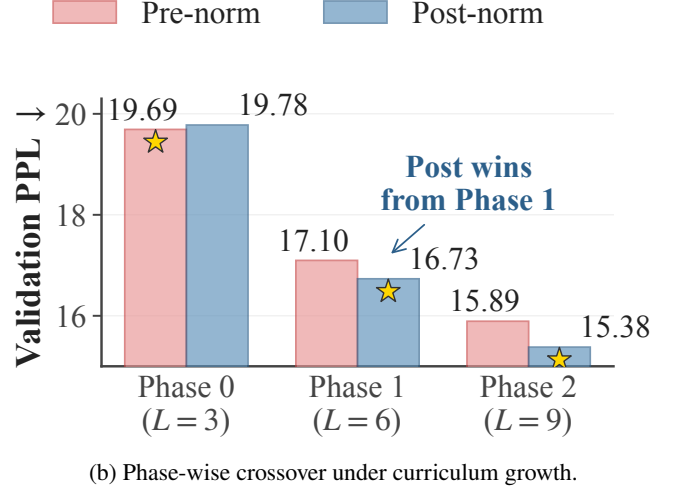
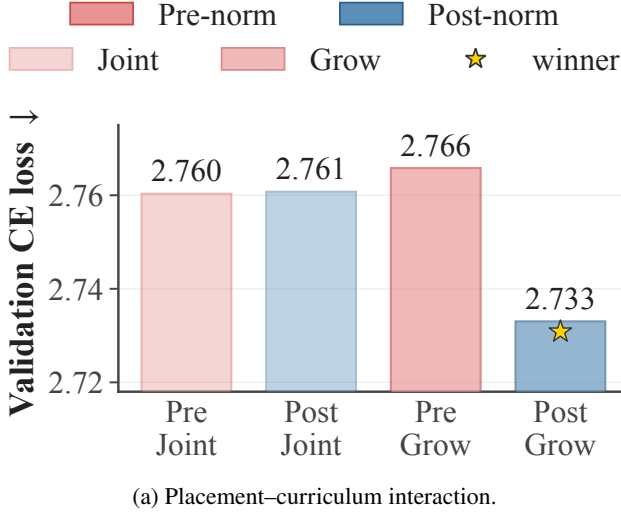


Figure 3: Main placement-curriculum results. The pre-post ranking changes under curriculum growth, and the crossover appears after the initial shallow phase.

2015; Jiao et al. 2020), the student copies the teacher embedding $\mathbf{E}$ and language-modeling head $\mathbf{W}_{lm}$. The embedding remains frozen, while the language-modeling head is optimized jointly with decoder layers initialized using the default Qwen3 module-wise scheme with initializer range 0.02. All students are distilled on the FineWeb-Edu 10BT split (Penedo et al. 2024). All reported primary runs use seed 42, a global batch size of 64, and a maximum sequence length of 8192 across all configurations and phases.

Compared Configurations. We compare four configurations formed by crossing normalization placement and depth-introduction protocol: $\{\text{pre-norm}, \text{post-norm}\} \times \{\text{joint}, \text{grow}\}$. In joint training, all blocks are active from the first update and are trained end-to-end. In curriculum growth, blocks are appended sequentially following Algorithm 1: each phase activates one additional block, carries over the previously trained weights, and introduces a newly initialized block when depth increases. Within each paired placement comparison, the teacher, student architecture, data order, objective, optimizer configuration, and initialization protocol are matched. Joint and grow then follow their respective activation, duration, replay, and reset schedules.

Training Budget. Each grow phase runs for 50,000 steps. Because sequences are variable-length and most documents fall well below the 8192-token cap, each phase processes approximately 3.11B non-padding tokens rather than the fully-packed upper bound. The three grow phases consume the same data shard, so grow sees 3.11B unique tokens repeated across phases (9.34B total). Joint training runs 50,000 steps over the same 3.11B-token shard. At every phase transition, the optimizer state is reset and the learning-rate schedule is restarted with warmup. Because growth activates fewer layers in early phases, we report a unique-token-matched primary comparison and active-layer-token controls; Appendix Table A2 summarizes the budget for each configuration. The

primary setting matches unique-token exposure, while the compute proxy matches student active-layer tokens, computed as training tokens multiplied by the number of active decoder layers.

Evaluation Metrics. The primary metric is validation cross-entropy (CE) loss $\downarrow$ on a fixed held-out set of 4,833 documents (approximately 4.98M tokens), evaluated every 2,000 steps, with perplexity (PPL) $\downarrow$ reported as an equivalent scale. For mechanism analysis, we measure block-boundary residual RMS across grow checkpoints and inspect validation-loss trajectories around phase transitions.

Main Results

For **RQ1**, we evaluate whether the relative behavior of pre-norm and post-norm changes across depth-introduction protocols. We compare the two placements under matched architecture, data, objective, and unique-token exposure; grow performs more optimization steps, examined through active-layer-token controls.

Placement-Curriculum Interaction. Table 1 and Figure 3a report the primary comparison. Under joint training, pre-norm and post-norm reach nearly identical validation losses, with CE values of 2.7603 and 2.7607, respectively. Under curriculum growth, the ranking changes: post-norm reduces validation CE from 2.7658 to 2.7330 compared with pre-norm. This yields $\Delta_{\text{joint}} = +0.0004$ and $\Delta_{\text{grow}} = -0.0328$, where $\Delta = \text{Post} - \text{Pre}$. The joint gap indicates how far the two placements drift apart when depth is held fixed, and the grow gap is roughly eighty times larger. The resulting interaction, $\Delta_{\text{grow}} - \Delta_{\text{joint}} = -0.0332$, quantifies the observed shift toward post-norm under curriculum growth in this distillation setting.

Phase-wise Crossover. The per-phase results further show where the ranking changes during curriculum growth. As

	Unique-token-matched	
	Joint	Grow
Pre-norm	2.7603	2.7658
Post-norm	2.7607	2.7330
Δ (Post–Pre)	+0.0004	–0.0328
Interaction		–0.0332

Table 1: Primary placement–curriculum interaction. Validation CE loss $\downarrow$ under unique-token-matched budgets. Δ = Post – Pre, so negative values indicate a post-norm advantage.

Norm	CE $\downarrow$	KD $\downarrow$	PPL $\downarrow$
Pre	2.9802	3.0155	19.6900
Post	2.9846	3.0183	19.7800

Table 2: Single-block Phase 0 results. CE, KD loss, and PPL are reported for one-block pre-norm and post-norm students.

shown in Figure 3b, pre-norm is slightly better in the shallow Phase 0, where the student contains only one block. After additional blocks are appended, post-norm becomes better in both Phase 1 and Phase 2. This crossover matches the expected shift from optimizing a shallow block to conditioning the boundary distribution passed to appended blocks. We next use single-block and active-layer-token controls to test whether shallow-block quality or longer optimization can explain the pattern.

Controlled Analysis

For **RQ2**, we examine whether the post-norm advantage under curriculum growth can be explained by simpler alternatives. We test two possibilities under the same held-out evaluation: post-norm may form stronger individual blocks, or curriculum growth may benefit from additional student-decoder active compute.

Single-Block Control. We first test whether post-norm is already stronger when training a single block in isolation. Table 2 reports Phase 0 results, where the student contains only one block ($L=3$). Pre-norm is marginally ahead on CE, KD loss, and PPL, with all gaps below 0.01 in CE/KD, which rules out post-norm forming a better shallow block as the source of its grow advantage; instead, the advantage appears only after additional blocks are appended, matching the phase-wise crossover in Figure 3b.

Active-Layer-Token Control. We next test whether additional student-decoder active compute explains the grow advantage. Because curriculum growth traverses three phases while joint training uses one full-depth phase, we extend post-joint to matched active-layer budgets; these controls use phase-aligned 50k-step restarts at fixed depth and continue onto fresh data rather than replaying the grow shard. Table 3 shows that post-joint improves from 2.7607 to 2.7518 when matched to grow on student active-layer tokens, and to 2.7428 when matched to grow on total training tokens while using

Run	Matches grow on	CE $\downarrow$
Post-Joint	Unique tokens	2.7607
Post-Joint-long	Active-layer tokens	2.7518
Post-Joint-longer	Total tokens	2.7428

Table 3: Post-joint budget controls. Validation CE loss $\downarrow$ under three matching criteria.

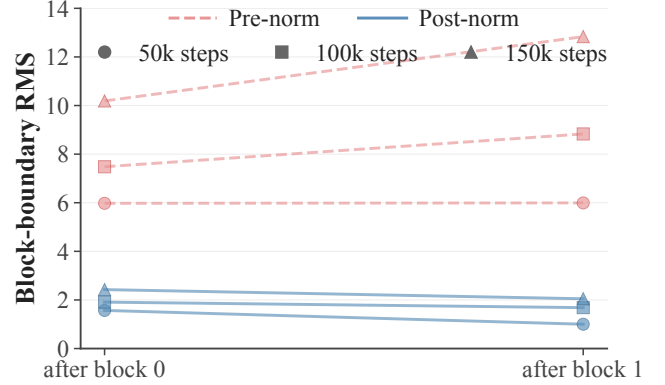


Figure 4: Block-boundary RMS across growth phases.

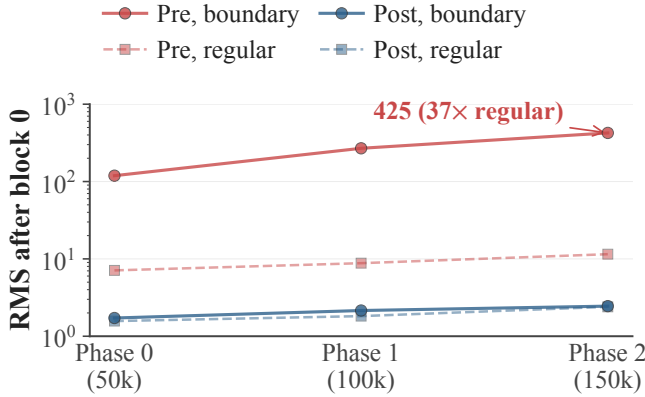
more student-decoder active compute. Both nonetheless remain worse than post-grow (2.7330), despite the additional compute. This suggests that the post-norm grow result is not reducible to additional student active-layer compute under the evaluated schedules.

Boundary-Scale Diagnostics

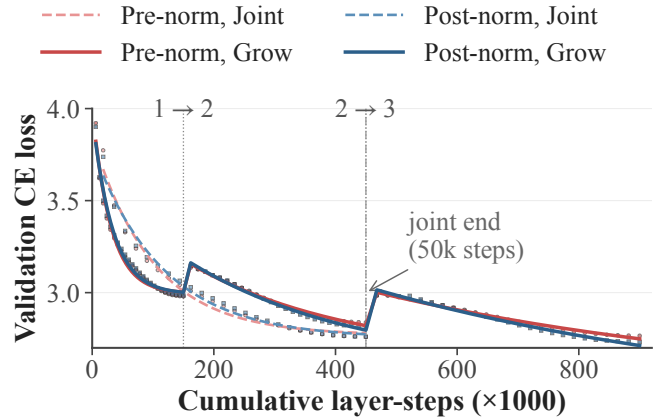
For **RQ3**, we test whether block-boundary dynamics support forward conditioning as a contributing mechanism. At each depth transition, a newly appended block receives the residual stream produced by the prefix trained in the preceding phase. We examine residual-stream scale and token composition, appended-block contributions, transition-localized loss, and the freeze control, which together characterize how the boundary distribution evolves as depth grows.

Block-Boundary Scale Drift. Figure 4 shows that post-norm keeps block-boundary RMS controlled, whereas pre-norm drifts as depth grows: after block 0, the per-token RMS standard deviation rises from 5.81 to 19.98 under pre-norm but remains approximately 0.07–0.19 under post-norm. On the final-phase diagnostic batch (Appendix Figure A1), pre-norm shows a heavy-tailed distribution (kurtosis > 2400) versus a far less heavy-tailed post-norm distribution.

Token-Type Attribution of Scale Drift. The extreme kurtosis suggests that pre-norm’s drift is driven by a small subset of tokens rather than a uniform shift. Figure 5a localizes pre-norm’s drift to structural tokens: document-boundary RMS grows from 119 to 425 ($37\times$ regular-token RMS at 150k), while regular-token RMS grows only from 7.1 to 11.5; post-norm keeps both below 2.5 (2.45 vs. 2.40), so the drift is concentrated at structural-token massive activations rather than a uniform global shift.



(a) Token-type attribution of scale drift.



(b) Validation-loss trajectories and grow transitions.

Figure 5: Forward-conditioning diagnostics. Post-norm exhibits limited structural-token scale drift, and the placement gap emerges after block-appending transitions.

Block-Removal Sensitivity and Representation Change.

On a fixed 2×2048 -token diagnostic batch (seed 42), we bypass each entire block and measure $\Delta CE_l = CE(\text{skip block } l) - CE(\text{full model})$, together with angular distance $\arccos(\cosine(\mathbf{h}_{in}, \mathbf{h}_{out}))/\pi$. For a post-norm three-layer block, bypass removes its attention and MLP modules together with six post-residual RMSNorm operations; the model entry norm remains outside the skipped block. Thus, ΔCE measures whole-block removal sensitivity rather than the attention and MLP contribution alone.

Under pre-norm growth, bypassing the final block changes CE by only 0.02, and its angular distance is 0.031, indicating a near-identity mapping. Removing any post-grow block instead raises CE by at least 2.38, with angular distances above 0.34; neither joint model has a near-identity final block. Post-joint places low removal sensitivity on block 0 ($\Delta CE = 0.55$) but high sensitivity on block 2 (4.00), showing that uneven allocation alone does not imply the pre-grow failure pattern. Thus, scale drift and a nearly identity-mapped final block coincide under pre-norm growth, while post-norm retains controlled scales and nontrivial representation changes across blocks.

Transition-Localized Loss Divergence. Figure 5b shows that the placement gap emerges after blocks are appended. Transition-aligned loss reveals that post-norm incurs larger immediate increases at both transitions, with much slower recovery after the first and only a small delay after the second (Appendix Table A4), whereas pre-norm changes more smoothly. The smooth pre-norm transition is consistent with a smaller initial perturbation, yet its final block is nearly identity-mapped; post-norm incurs a larger adjustment but develops nontrivial block changes. Thus, boundary-scale control should not be interpreted as a smoother immediate transition: transition smoothness and final block utilization need not coincide in general.

Freeze Control. Finally, we freeze completed blocks and update only the newly appended block under matched bud-

Run	Metric	Block 0	Block 1	Block 2
Pre-Grow	$\Delta CE \uparrow$	8.13	2.36	0.02
	Angular dist.	0.499	0.310	0.031
Post-Grow	$\Delta CE \uparrow$	8.32	3.74	2.38
	Angular dist.	0.483	0.365	0.342
Pre-Joint	$\Delta CE \uparrow$	8.05	1.47	2.34
	Angular dist.	0.485	0.245	0.360
Post-Joint	$\Delta CE \uparrow$	0.55	0.99	4.00
	Angular dist.	0.395	0.166	0.437

Table 4: Block-removal sensitivity and representation change. ΔCE includes removal of internal normalization; angular distance measures input–output change. Only Pre-Grow has a near-identity final block ($\Delta CE \approx 0$, angular ≈ 0.03).

gets and transition policies. Freezing worsens both placements by approximately 0.144 CE but leaves the gap nearly unchanged (-0.0328 vs. -0.0332), so retraining earlier blocks does not account for the placement gap (Appendix Table A3). These diagnostics observe rather than intervene on the boundary distribution, and together they provide converging evidence for boundary-scale conditioning.

Conclusion

Pre-norm and post-norm are nearly tied under joint training, whereas post-norm is better under curriculum depth growing. Single-block, compute-matched, freeze, and boundary diagnostics associate this crossover with controlled boundary scales, ruling out shallow-block quality, active-layer compute, or retraining alone. The results therefore motivate treating normalization placement and depth curriculum as coupled rather than independent design choices.

References

- Ainslie, J.; Lee-Thorp, J.; de Jong, M.; Zemlyanskiy, Y.; Lebrón, F.; and Sanghai, S. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In Bouamor, H.; Pino, J.; and Bali, K., eds., *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, 4895–4901. Association for Computational Linguistics.
- Ba, L. J.; Kiros, J. R.; and Hinton, G. E. 2016. Layer Normalization. *CoRR*, abs/1607.06450.
- Bachlechner, T.; Majumder, B. P.; Mao, H. H.; Cottrell, G.; and McAuley, J. J. 2021. ReZero is all you need: fast convergence at large depth. In de Campos, C. P.; Maathuis, M. H.; and Quaeghebeur, E., eds., *Proceedings of the Thirty-Seventh Conference on Uncertainty in Artificial Intelligence, UAI 2021, Virtual Event, 27-30 July 2021*, volume 161 of *Proceedings of Machine Learning Research*, 1352–1361. AUAI Press.
- Chen, T.; Goodfellow, I. J.; and Shlens, J. 2016. Net2Net: Accelerating Learning via Knowledge Transfer. In Bengio, Y.; and LeCun, Y., eds., *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*.
- Ding, M.; Yang, Z.; Hong, W.; Zheng, W.; Zhou, C.; Yin, D.; Lin, J.; Zou, X.; Shao, Z.; Yang, H.; and Tang, J. 2021. CogView: Mastering Text-to-Image Generation via Transformers. In Ranzato, M.; Beygelzimer, A.; Dauphin, Y. N.; Liang, P.; and Vaughan, J. W., eds., *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, 19822–19835.
- Du, W.; Luo, T.; Qiu, Z.; Huang, Z.; Shen, Y.; Cheng, R.; Guo, Y.; and Fu, J. 2024. Stacking Your Transformers: A Closer Look at Model Growth for Efficient LLM Pre-Training. In Globersons, A.; Mackey, L.; Belgrave, D.; Fan, A.; Paquet, U.; Tomczak, J. M.; and Zhang, C., eds., *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.
- Elhoushi, M.; Shrivastava, A.; Liskovich, D.; Hosmer, B.; Wasti, B.; Lai, L.; Mahmoud, A.; Acun, B.; Agarwal, S.; Roman, A.; Aly, A. A.; Chen, B.; and Wu, C. 2024. LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding. In Ku, L.; Martins, A.; and Srikumar, V., eds., *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, 12622–12642. Association for Computational Linguistics.
- Gou, J.; Yu, B.; Maybank, S. J.; and Tao, D. 2021. Knowledge Distillation: A Survey. *Int. J. Comput. Vis.*, 129(6): 1789–1819.
- Gu, X.; Liu, L.; Yu, H.; Li, J.; Chen, C.; and Han, J. 2021. On the Transformer Growth for Progressive BERT Training. In Toutanova, K.; Rumshisky, A.; Zettlemoyer, L.; Hakkani-Tür, D.; Beltagy, I.; Bethard, S.; Cotterell, R.; Chakraborty, T.; and Zhou, Y., eds., *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021*, 5174–5180. Association for Computational Linguistics.
- Hinton, G. E.; Vinyals, O.; and Dean, J. 2015. Distilling the Knowledge in a Neural Network. *CoRR*, abs/1503.02531.
- Huang, G.; Sun, Y.; Liu, Z.; Sedra, D.; and Weinberger, K. Q. 2016. Deep Networks with Stochastic Depth. In Leibe, B.; Matas, J.; Sebe, N.; and Welling, M., eds., *Computer Vision - ECCV 2016 - 14th European Conference, Amsterdam, The Netherlands, October 11-14, 2016, Proceedings, Part IV*, volume 9908 of *Lecture Notes in Computer Science*, 646–661. Springer.
- Huang, X. S.; Pérez, F.; Ba, J.; and Volkovs, M. 2020. Improving Transformer Optimization Through Better Initialization. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, volume 119 of *Proceedings of Machine Learning Research*, 4475–4483. PMLR.
- Jiao, X.; Yin, Y.; Shang, L.; Jiang, X.; Chen, X.; Li, L.; Wang, F.; and Liu, Q. 2020. TinyBERT: Distilling BERT for Natural Language Understanding. In Cohn, T.; He, Y.; and Liu, Y., eds., *Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020*, volume EMNLP 2020 of *Findings of ACL*, 4163–4174. Association for Computational Linguistics.
- Kan, K.; Li, X.; Zhang, B. J.; Sahai, T.; Osher, S. J.; Kumar, K.; and Katsoulakis, M. A. 2025. Stability of Transformers under Layer Normalization. *CoRR*, abs/2510.09904.
- Kim, J.; Lee, B.; Park, C.; Oh, Y.; Kim, B.; Yoo, T.; Shin, S.; Han, D.; Shin, J.; and Yoo, K. M. 2025. Peri-LN: Revisiting Normalization Layer in the Transformer Architecture. In Singh, A.; Fazel, M.; Hsu, D.; Lacoste-Julien, S.; Berkenkamp, F.; Maharaj, T.; Wagstaff, K.; and Zhu, J., eds., *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, volume 267 of *Proceedings of Machine Learning Research*. PMLR / OpenReview.net.
- Li, T.; Han, D.; Cao, Z.; Huang, H.; Zhou, M.; Chen, M.; Zhao, E.; Jiang, X.; Jiang, G.; and Huang, G. 2026. SiameNorm: Breaking the Barrier to Reconciling Pre/Post-Norm. *CoRR*, abs/2602.08064.
- Liu, L.; Liu, X.; Gao, J.; Chen, W.; and Han, J. 2020. Understanding the Difficulty of Training Transformers. In Webber, B.; Cohn, T.; He, Y.; and Liu, Y., eds., *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16-20, 2020*, 5747–5763. Association for Computational Linguistics.
- Loshchilov, I.; Hsieh, C.; Sun, S.; and Ginsburg, B. 2025. nGPT: Normalized Transformer with Representation Learning on the Hypersphere. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net.
- Men, X.; Xu, M.; Zhang, Q.; Yuan, Q.; Wang, B.; Lin, H.; Lu, Y.; Han, X.; and Chen, W. 2025. ShortGPT: Layers in Large Language Models are More Redundant Than You Expect. In Che, W.; Nabende, J.; Shutova, E.; and Pilehvar, M. T., eds.,

- Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, volume ACL 2025 of *Findings of ACL*, 20192–20204. Association for Computational Linguistics.
- Penedo, G.; Kydřicek, H.; Allal, L. B.; Lozhkov, A.; Mitchell, M.; Raffel, C. A.; von Werra, L.; and Wolf, T. 2024. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. In Globersons, A.; Mackey, L.; Belgrave, D.; Fan, A.; Paquet, U.; Tomczak, J. M.; and Zhang, C., eds., *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.
- Sanh, V.; Debut, L.; Chaumond, J.; and Wolf, T. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *CoRR*, abs/1910.01108.
- Shazeer, N. 2020. GLU Variants Improve Transformer. *CoRR*, abs/2002.05202.
- Su, J.; Ahmed, M. H. M.; Lu, Y.; Pan, S.; Bo, W.; and Liu, Y. 2024. RoFormer: Enhanced transformer with Rotary Position Embedding. *Neurocomputing*, 568: 127063.
- Sun, Z.; Yu, H.; Song, X.; Liu, R.; Yang, Y.; and Zhou, D. 2020. MobileBERT: a Compact Task-Agnostic BERT for Resource-Limited Devices. In Jurafsky, D.; Chai, J.; Schluter, N.; and Tetreault, J. R., eds., *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*, 2158–2170. Association for Computational Linguistics.
- Team, Q. 2025. Qwen3 Technical Report. *CoRR*, abs/2505.09388.
- Touvron, H.; Lavril, T.; Izacard, G.; Martinet, X.; Lachaux, M.; Lacroix, T.; Rozière, B.; Goyal, N.; Hambro, E.; Azhar, F.; Rodriguez, A.; Joulin, A.; Grave, E.; and Lample, G. 2023. LLaMA: Open and Efficient Foundation Language Models. *CoRR*, abs/2302.13971.
- Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, L.; and Polosukhin, I. 2017. Attention is All you Need. In Guyon, I.; von Luxburg, U.; Bengio, S.; Wallach, H. M.; Fergus, R.; Vishwanathan, S. V. N.; and Garnett, R., eds., *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, 5998–6008.
- Wang, H.; Ma, S.; Dong, L.; Huang, S.; Zhang, D.; and Wei, F. 2024. DeepNet: Scaling Transformers to 1,000 Layers. *IEEE Trans. Pattern Anal. Mach. Intell.*, 46(10): 6761–6774.
- Wang, W.; Wei, F.; Dong, L.; Bao, H.; Yang, N.; and Zhou, M. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In Larochelle, H.; Ranzato, M.; Hadsell, R.; Balcan, M.; and Lin, H., eds., *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*.
- Wei, T.; Wang, C.; Rui, Y.; and Chen, C. W. 2016. Network Morphism. In Balcan, M.; and Weinberger, K. Q., eds., *Proceedings of the 33rd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016*, volume 48 of *JMLR Workshop and Conference Proceedings*, 564–572. JMLR.org.
- Wu, C.; Gan, Y.; Ge, Y.; Lu, Z.; Wang, J.; Feng, Y.; Shan, Y.; and Luo, P. 2024. LLaMA Pro: Progressive LLaMA with Block Expansion. In Ku, L.; Martins, A.; and Srikumar, V., eds., *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, 6518–6537. Association for Computational Linguistics.
- Xia, M.; Gao, T.; Zeng, Z.; and Chen, D. 2024. Sheared LLaMA: Accelerating Language Model Pre-training via Structured Pruning. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.
- Xiong, R.; Yang, Y.; He, D.; Zheng, K.; Zheng, S.; Xing, C.; Zhang, H.; Lan, Y.; Wang, L.; and Liu, T. 2020. On Layer Normalization in the Transformer Architecture. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, volume 119 of *Proceedings of Machine Learning Research*, 10524–10533. PMLR.
- Zhang, B.; and Sennrich, R. 2019. Root Mean Square Layer Normalization. In Wallach, H. M.; Larochelle, H.; Beygelzimer, A.; d’Alché-Buc, F.; Fox, E. B.; and Garnett, R., eds., *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, 12360–12371.
- Zhang, H.; Dauphin, Y. N.; and Ma, T. 2019. Fixup Initialization: Residual Learning Without Normalization. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net.
- Zheng, C.; Sun, J.; Gao, Y.; Wang, C.; Wang, Y.; Xiong, J.; Ren, L.; Peng, B.; Wang, Q.; Shang, X.; Schwager, M.; Schneider, A.; Nevmyvaka, Y.; and Liu, X. 2026. GeoNorm: Unify Pre-Norm and Post-Norm with Geodesic Optimization. *CoRR*, abs/2601.22095.
- Zhuo, Z.; Zeng, Y.; Wang, Y.; Zhang, S.; Yang, J.; Li, X.; Zhou, X.; and Ma, J. 2025. HybridNorm: Towards Stable and Efficient Transformer Training via Hybrid Normalization. *CoRR*, abs/2503.04598.

Appendix A: Training and Compute Protocol

A.1 Detailed Hyperparameter Settings

Table A1 lists shared hyperparameters. Training uses AdamW with cosine decay and warmup.

A.2 Budget and Compute Protocol

Because curriculum growth activates fewer layers in early phases, step-matched budgets are not compute-fair. We use a unique-token-matched primary comparison and additional post-joint active-layer-token controls:

- **Unique-token-matched:** joint and grow see the same *unique* tokens. This compares quality at equal data coverage, although grow uses more active-layer compute across three phases.
- **Active-layer-token control:** post-joint is extended to the active-layer-token budget of grow, i.e. trained tokens multiplied by the number of active decoder layers at each step. We additionally report a $3\times$ post-joint run beyond that proxy-matched budget.

Compute can be approximated by active-layer tokens:

$$\text{active-layer tokens} = \sum_{\text{step } t} (\text{tokens at } t) \cdot (\text{active layers at } t).$$

Hyperparameter	Value
Optimizer	AdamW
Optimizer β_1, β_2	0.9, 0.95
Optimizer ϵ	1e-8
Weight Decay	0
Peak Learning Rate	1e-4
Minimum Learning Rate	1e-6
Learning Rate Schedule	Cosine decay with warmup
Warmup Steps	$0.015 \times 50,000 = 750$
Gradient Clipping	0.5 (global L2 norm)
Training Precision	BF16
Global Batch Size	64
Sequence Length	8192 (max)
Evaluation Interval	2,000 steps
Distillation Weight α	0.7
Language Modeling Weight γ	0.3
Temperature τ	1.0
Dropout	0
Attention Dropout	0
Seed	42

Table A1: Hyperparameters shared across the primary 2×2 configurations. Controls additionally vary training duration or trainable blocks as stated in their protocols.

A.3 Phase Transition Optimization Reset

At each curriculum growth transition, we reset the optimizer state and restart the learning-rate schedule with warmup for two reasons:

1. **Depth increase.** Moving from p to $p + 1$ blocks introduces newly initialized weights. Restarting the LR schedule with warmup updates these layers smoothly instead of exposing them to stale momentum.

Config	Phases	Layers	Total tok.	Unique tok.
Joint-50k	1	9	3.11B	3.11B
Grow- $3 \times 50k$	3	$3 \rightarrow 6 \rightarrow 9$	9.34B	3.11B
Joint (proxy-matched)	1	9	6.22B	6.22B
Joint-150k	1	9	9.34B	9.34B

Table A2: Training budgets. Grow reuses one fixed shard, whereas longer joint controls continue onto fresh data; proxy-matched joint matches student active-layer tokens, and Joint-150k is the $3\times$ stress test.

2. **Stale gradient statistics.** Resetting Adam momentum prevents statistics accumulated in the shallower phase from biasing the deeper network.

Thus, each phase inherits model weights but re-estimates optimizer statistics rather than carrying shallower-phase momentum across successive transitions.

Appendix B: Additional Analyses

B.1 Freeze Ablation: Probing Forward Conditioning

Setup. The main grow protocol retraines all active blocks in every phase. A natural alternative is to *freeze* each block once its phase completes, so that phase p trains only the newly appended block p while blocks $0, \dots, p - 1$ are held fixed. Freeze and retrain differ in two ways: (i) freeze gives weaker absolute performance because earlier blocks cannot adapt to the deeper stack, and (ii) under freeze the input distribution to a newly appended block is *strictly fixed* (it is the exact output of frozen blocks), which sharpens the forward-conditioning channel. If retraining primarily compensated for pre-norm’s boundary drift, the post-pre gap would be expected to amplify under freeze. Instead, the observed gap is essentially unchanged. However, freeze does not fully isolate the forward channel: the newly appended block is still trained, so its backward-gradient dynamics at the append step still differ across placements. The result is therefore compatible with a forward-conditioning contribution, but does not exclude a backward-at-append contribution.

Protocol. The freeze variant follows the grow protocol of the main paper exactly, except that at the start of phase $p + 1$ the parameters of blocks $0, \dots, p$ are frozen (no gradient, no optimizer state); only block $p + 1$ receives updates. Step budgets, LR schedule, data shards, and the reset-at-transition policy are identical to grow. Pre-norm and post-norm use the same paired seed and initialization as the main runs.

Result. Table A3 reports validation CE at 150k. Freeze is uniformly worse than retrain by ≈ 0.144 CE for both placements, showing that retraining earlier blocks is beneficial in absolute terms. The placement gap, however, is essentially unchanged: $\Delta_{\text{grow}} = -0.0328$ versus $\Delta_{\text{freeze}} = -0.0332$, a difference of -0.0004 ($\approx 1.2\%$ of the gap). On PPL the gap is wider under freeze (-0.60 vs -0.51), but PPL is the exponential of CE, so this amplification is consistent with an unchanged CE gap.

	Grow (retrain)	Freeze
Pre-norm	2.7658	2.9104
Post-norm	2.7330	2.8772
Δ (Post–Pre)	–0.0328	–0.0332
$\Delta_{\text{freeze}} - \Delta_{\text{grow}}$	–0.0004	

Table A3: Freeze ablation. Validation CE loss $\downarrow$ at 150k under the unique-token-matched budget (grow and freeze both traverse three 50k-step phases over the same shard). Grow reproduces the main interaction (retrain); freeze holds previously appended blocks fixed. Δ = Post – Pre (negative = post wins). The placement gap is essentially unchanged across retrain/freeze ($|\Delta|$ differs by 0.0004).

Metric	50k (Pre/Post)	100k (Pre/Post)
Loss jump	0.0226 / 0.2500	0.0230 / 0.2847
AUC _{1k}	0.0694 / 0.1820	0.0688 / 0.2059
AUC _{5k}	0.1217 / 0.1464	0.1277 / 0.1774
Recovery steps	101 / 19,914	19,054 / 19,426

Table A4: Transition-aligned training-loss response at the 50k and 100k block-appending steps. Jump and AUC are measured relative to the pre-transition baseline; recovery is the first step at which a 200-step smoothed loss returns to that baseline. Post-norm pays a larger immediate adjustment cost despite its better final grow loss.

Interpretation. The result is consistent with the forward-conditioning account, with two caveats. First, the absolute freeze penalty (≈ 0.144 CE) is nearly identical across placements, so a retraining-only explanation for post-norm’s relative advantage is unlikely under this control. Second, the gap’s near-invariance to retrain/freeze is consistent with the RQ3 RMS evidence: pre-norm’s boundary drift is already present in the outputs of the prefix at the append step, whether or not those blocks are retrained later. The result therefore weighs against the stronger claim that retraining actively *masks* pre-norm’s drift, and suggests that later joint updates do not recover the full append-time cost. We emphasize what freeze does *not* establish: because the newly appended block remains trainable, its backward-gradient dynamics at the append step still differ by placement, so the gap’s stability is also compatible with a backward-at-append channel. Forward conditioning is therefore a candidate explanation for post-norm’s grow advantage, but isolating it as the sole cause would require an intervention that changes the forward boundary distribution without changing backward updates.

B.2 Mechanism Diagnostics

The main paper reports the primary interaction and phase-wise crossover in Figure 3, and the block-boundary RMS trend, token-type attribution, and transition-localized loss trajectories in Figures 4 and 5. Figures A1 and A2 complement those aggregate trends with the final-checkpoint RMS distribution and per-block deletion behavior.

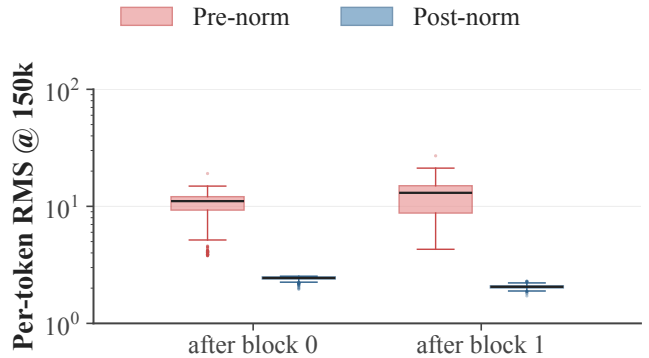


Figure A1: Final-phase per-token RMS distributions.

B.3 Additional Protocol Checks

Data-Iterator Reset. Within curriculum growth, we compared resetting the data iterator at phase boundaries with continuing from its current position. Both variants draw from the same fixed 3.11B-token shard and wrap within that shard after reaching its end; a phase transition never switches to new data. They produce similar final losses and the same placement ranking. The main results use the no-reset variant, so a phase change continues from the current offset in this repeating cycle rather than explicitly rewinding it, while unique-token coverage remains 3.11B. The longer joint controls instead continue onto fresh data, so their total- and unique-token counts coincide.

Joint Learning-Rate Restarts. The 100k and 150k post-joint controls in Table A2 keep all nine decoder layers active and restart the 50k-step learning-rate schedule at the same boundaries used by grow. Each restart produces a transient loss increase followed by renewed descent, but the subsequent decrease is slower than the corresponding curriculum-growth trajectory. Their endpoint CE values improve to 2.7518 and 2.7428, respectively, yet remain above post-grow at 2.7330; LR restarting alone therefore does not reproduce the grow trajectory.

Appendix C: Implementation Details

Model and Systems. The Qwen3-8B-Base teacher has 36 decoder layers. The nine-layer student uses hidden size 4096, 32 attention heads, 8 key–value heads, and intermediate size 12288. It contains approximately 3.09B parameters including the teacher-initialized embedding and language-modeling head, 2.47B when excluding the head but retaining the embedding, and 1.85B in the decoder stack alone. The embedding is frozen, whereas the language-modeling head is trained. Training uses FSDP over 8 GPUs, while teacher inference uses a separate 8-GPU pool; gradient checkpointing and MLP recomputation are enabled.

Data Processing and Evaluation. Examples are tokenized independently, truncated to 8192 tokens, and right-padded to the longest sequence in each batch; multiple documents are not packed into one sequence. Padding is excluded from

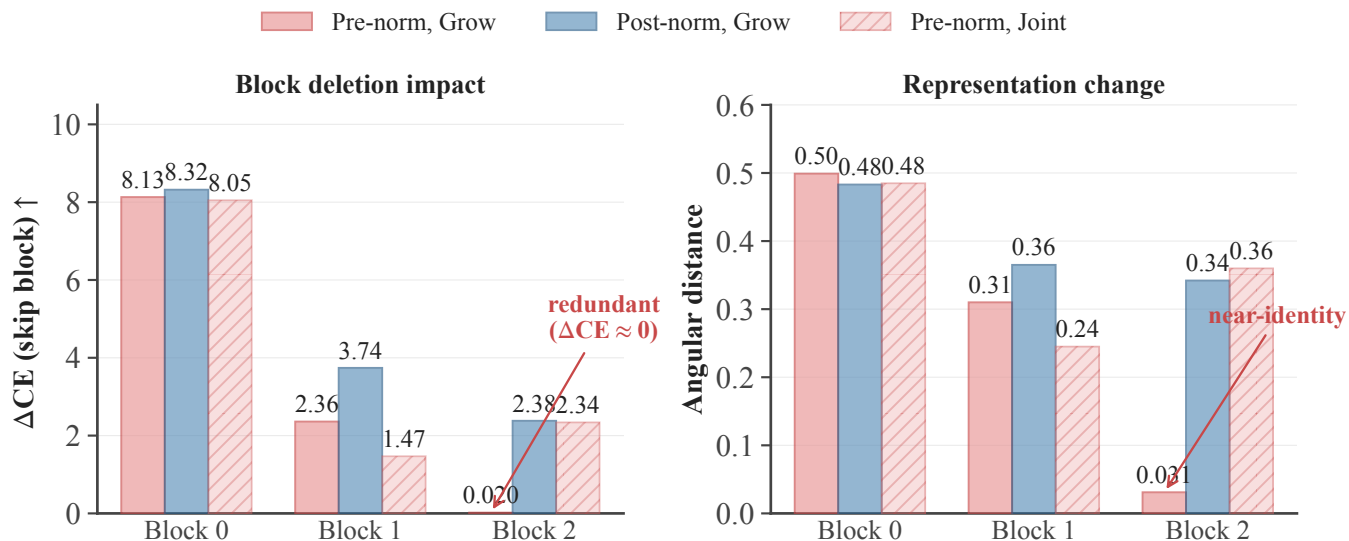


Figure A2: Whole-block removal sensitivity and representation change. Whole-block bypass also removes internal normalization operations.

the token accounting and loss. The fixed validation set contains 4,833 documents and approximately 4.98M tokens, and evaluation is run every 2,000 optimization steps. Main tables report the scheduled endpoints rather than selecting the best validation checkpoint.